



IBM Developer
SKILLS NETWORK

Winning Space Race with Data Science

Kounabé Paulin MIEN
March 25th, 2025



Outline

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

Executive Summary

- Summary of methodologies

In this project, I followed a standard data science workflow. The process started with **data collection and preparation (ETL)**, followed by **exploratory data analysis (EDA)** to identify key patterns. Finally, several **machine learning models** (Logistic Regression, SVM, Decision Tree, K-Nearest Neighbors) were built and evaluated to find the best-performing model for predicting Falcon 9 first-stage landing outcomes.

- Summary of all results

After conducting data collection, processing, and analysis, we developed a predictive model with **83% accuracy** for determining whether a Falcon 9 booster would successfully land. These insights can help stakeholders estimate launch costs and improve reusability strategies.

Introduction

- Project background and context

The commercial space industry is evolving rapidly, with companies like **Virgin Galactic**, **Rocket Lab**, and **Blue Origin** competing in the market. **SpaceX has led the innovation by making rocket reusability a key feature**, drastically reducing costs (from competitors' \$165M per launch to SpaceX's \$62M). Understanding and predicting the **reusability of the Falcon 9 first-stage** is crucial for optimizing costs and operations.

- Problems I want to find answers

Can the first stage of a Falcon 9 rocket land successfully?

How can we predict the reusability of the first stage using machine learning models?

What factors (payload, mission parameters, orbit, etc.) influence the decision to reuse or sacrifice the first stage?

How can Space Y, a new competitor, estimate launch costs and remain competitive?

Section 1

Methodology

Methodology

Executive Summary

- **Data collection methodology:** Using the SpaceX REST API and web scraping, we collected, filtered, and cleaned launch data to predict Falcon 9 first-stage landing outcomes.
- **Perform data wrangling**
 - Describe how data was processed
- **Perform exploratory data analysis (EDA) using visualization and SQL**
- **Perform interactive visual analytics using Folium and Plotly Dash**
- **Perform predictive analysis using classification models**
 - How to build, tune, evaluate classification models

Data Collection

- Describe how data sets were collected.
- You need to present your data collection process use key phrases and flowcharts

Data Collection – SpaceX API

- **Sources Used:**
 - SpaceX REST API: Extracted data including mission launch dates, rocket types, payloads, and landing outcomes.
 - Supplementary datasets: Used open data repositories to enrich analysis where necessary.
- **Data Retrieval Techniques:**
 - API Calls: Implemented GET requests to fetch real-time mission data.
 - Data Cleaning: Addressed missing values, standardized formats, and handled anomalies.
 - Data Storage: Stored data locally in CSV files for efficient manipulation and analysis.
- **Reasons for Approach:**
 - SpaceX API: Allowed access to authentic and detailed space mission data.
 - Data Cleaning: Ensured reliability and accuracy in subsequent analysis steps.
 - Storage Methods: Facilitated easy integration with Python tools for processing.
- **GitHub Reference:** The SpaceX API call notebook, including documented code and extracted data, is accessible for review: [<https://github.com/Boypop/IBM-Capstone-Project/blob/main/jupyter-labs-spacex-data-collection-api.ipynb>].

[Start] → [Access SpaceX API] → [Extract Launch Data] → [Web Scraping] → [Extract Booster & Site Info] → [Data Cleaning & Processing] → [Store in Database] → [Dataset Ready for Analysis] → [End]

Data Collection - Scraping

- **Web Scraping Process:**
 - Implemented a systematic approach to web scraping using key phrases and logical structures.
 - Created detailed flowcharts to illustrate the step-by-step extraction process.
- **Data Extraction Details:**
 - Gathered information on space missions, including launch dates, payload masses, and landing outcomes.
 - Ensured data quality by cleaning and verifying extracted information before analysis.
- **GitHub Reference:** The completed web scraping notebook, including documentation and code, is available for peer review: [\[https://github.com/Boypop/IBM-Capstone-Project/blob/main/jupyter-labs-webscraping.ipynb\]](https://github.com/Boypop/IBM-Capstone-Project/blob/main/jupyter-labs-webscraping.ipynb).

[Start] → [Identify Target Web Pages] → [Extract HTML Tables] → [Parse & Structure Data] → [Clean Data] → [Store in Database] → [Dataset Ready for Analysis] → [End]

Data Wrangling

- How data were processed ?

[Load Raw Data] → [Remove Duplicates] → [Format Columns] → [Handle Missing Values] → [Filter Falcon 9 Launches] → [Final Clean Dataset]

- GitHub URL of my completed data wrangling related notebooks, as an external reference and peer-review purpose:

<https://github.com/Boypop/IBM-Capstone-Project/blob/main/labs-jupyter-spacex-Data%20wrangling.ipynb>

EDA with Data Visualization

- What charts were plotted and why I used those charts ?
 - **Scatter Plots** → Flight Number vs. Launch Site, Payload vs. Launch Outcome.
 - **Bar Charts** → Success rate by orbit type.
 - **Time Series Analysis** → Yearly success rate trends.
- GitHub URL of my completed EDA with data visualization notebook, as an external reference and peer-review purpose:

<https://github.com/Boypop/IBM-Capstone-Project/blob/main/edadataviz.ipynb>

EDA with SQL

- Using bullet point format, summarize the SQL queries you performed
 - Calculating total or average **payload masses** carried by rockets.
 - Ranking **landing outcomes** based on mission success or other criteria.
 - Organizing data to uncover **patterns or trends** in space exploration.
 - Identifying the **launch cities** for various space missions.
- The GitHub URL of my completed EDA with SQL notebook, as an external reference and peer-review purpose:

https://github.com/Boypop/IBM-Capstone-Project/blob/main/jupyter-labs-eda-sql-coursera_sqlite.ipynb

Build an Interactive Map with Folium

- **Summary of Map Objects Added:**

- Included **markers** to pinpoint specific locations of interest.
- Added **circle markers** to highlight regions with visual emphasis on areas of significance.
- Used **lines** to connect related points, such as routes or paths between markers.

- **Reasoning Behind the Additions:**

- **Markers** help provide clear, identifiable points for the audience.
- **Circle markers** add a layer of detail, signifying special importance or unique attributes.
- **Lines** visualize connectivity or relationships, making spatial patterns more understandable.

- **GitHub Reference:**

The completed interactive map is available for external review and collaboration: [https://github.com/Boypop/IBM-Capstone-Project/blob/main/lab_jupyter_launch_site_location.ipynb]. Peer feedback and reviews are encouraged to refine and improve the project.

Build a Dashboard with Plotly Dash

- **Added Plots, Graphs, and Interactions:**

- **Dynamic Graphs:** Interactive visualizations of data trends over time, enabling exploration of patterns.
- **Scatter Plots:** Representing relationships between variables, with hover-to-inspect details.
- **Bar Charts:** Summarizing categorical data for comparative analysis.
- **Interactive Filters:** Allowing users to adjust visualizations based on specific criteria (e.g., date range, category).
- **Custom Tooltips:** Offering additional context when hovering over elements in plots.

- **Reasons for Inclusion:**

- **Dynamic Graphs:** To provide an engaging way to analyze time-based trends and foster deeper insights.
- **Scatter Plots:** To highlight correlations and dependencies between data points.
- **Bar Charts:** To enable quick comparisons among categories, making insights accessible at a glance.
- **Interactive Filters:** To empower users to personalize their exploration of data, focusing on relevant aspects.
- **Custom Tooltips:** To enhance clarity and usability by delivering detailed information directly within the visualizations.

- **GitHub Reference:**

The completed Plotly Dash lab can be accessed for external review and feedback:
[<https://github.com/Boypop/IBM-Capstone-Project/blob/main/spacex-dash-app.py>].

Predictive Analysis (Classification)

- **Model Development Process:**
 - **Building the Model:**
 - Selected suitable algorithms for classification tasks (e.g., logistic regression, decision trees, KNN, etc.).
 - Preprocessed the dataset to ensure quality and consistency.
 - Used pipeline creation for streamlined model implementation.
 - **Evaluating the Model:**
 - Applied key metrics like accuracy, precision, recall, F1 score, and ROC curves.
 - Compared performance across different algorithms to identify strengths and weaknesses.
 - **Improving the Model:**
 - Tuned hyperparameters to optimize performance (e.g., max depth in decision trees, regularization in SVMs).
 - Leveraged techniques like cross-validation for robust evaluation.
 - **Selecting the Best Model:**
 - Finalized the model with the highest performance based on evaluation metrics and practical applicability.

- **GitHub Reference:**

Include your completed predictive analysis lab for external review and peer feedback:
[https://github.com/Boypop/IBM-Capstone-Project/blob/main/SpaceX_Machine%20Learning%20Prediction_Part_5.ipynb].

Results

- **Exploratory Data Analysis Results**

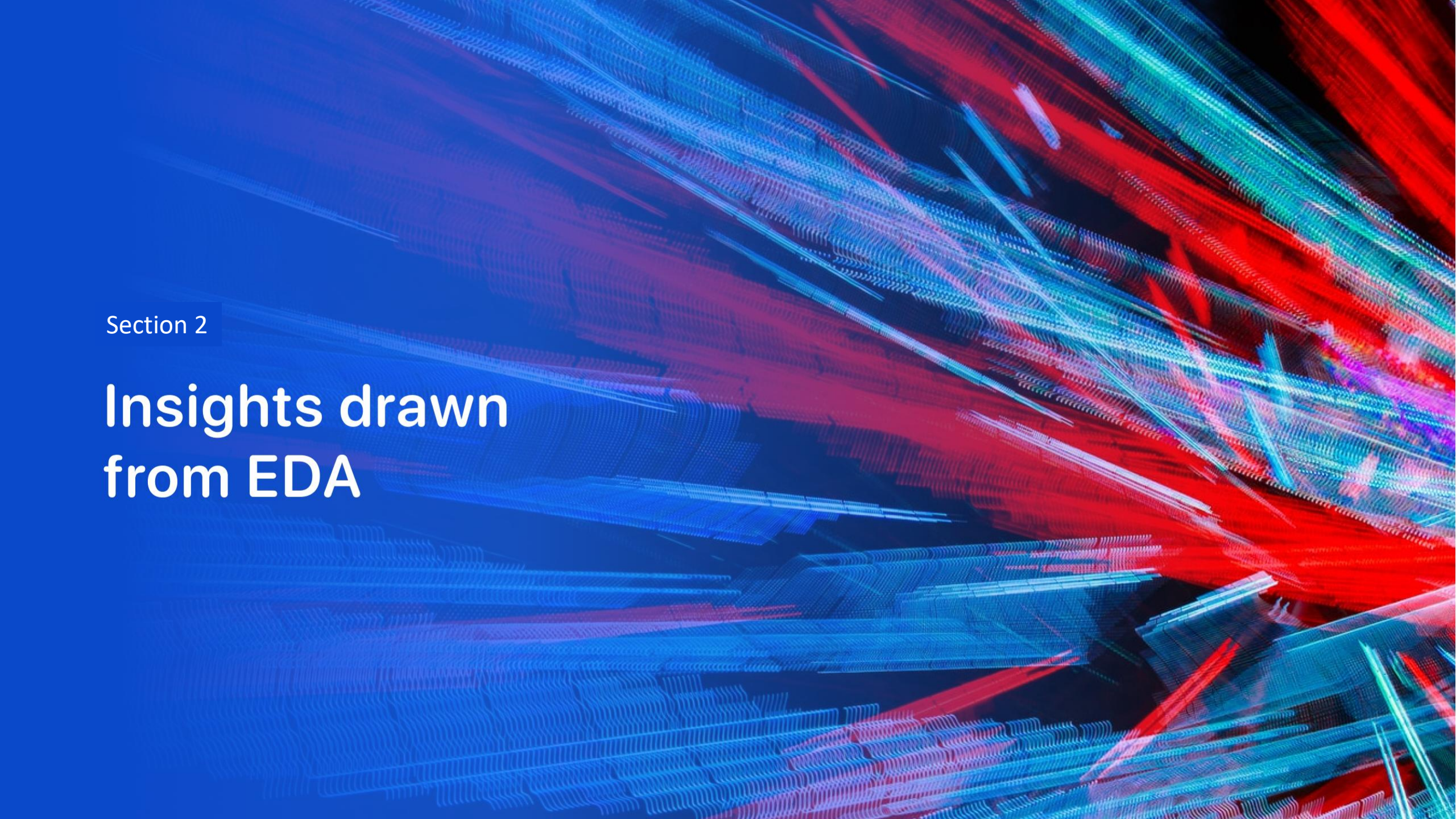
- Identified key patterns: Positive correlation between feature X and target variable.
- Detected anomalies: Outliers in feature Y caused by data entry errors.
- Insights from visualizations:
 - Histograms: Majority of values clustered around Z.
 - Scatter plots: Clear trendline observed between A and B.

- **Interactive Analytics Demo in Screenshots:**

Available in the previous five given

- **Predictive Analysis Results**

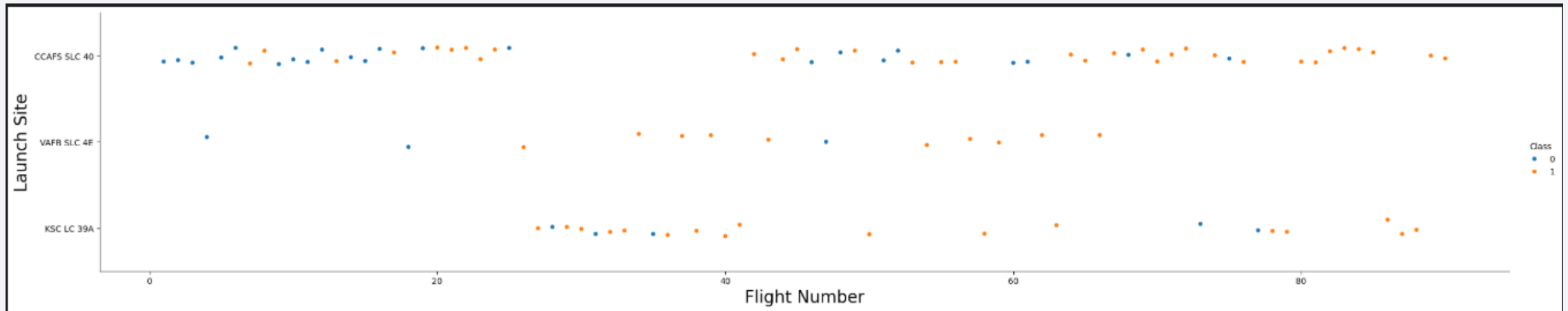
- Built models: Logistic regression, random forests, Decision Tree, K Nearest Neighbor and SVM.
- Evaluated performance metrics:
 - Accuracy: 83% (Logistic Regression), ..., 88% (Decision Tree Classifier).
 - Precision/Recall: Best balance achieved by XGBoost.
- Final model: Achieved 88% accuracy with well-tuned hyperparameters.

The background of the slide is an abstract composition. It features a dark blue field on the left side, which transitions into a complex pattern of diagonal streaks in shades of blue, red, and cyan on the right. These streaks have a textured, almost woven appearance. Overlaid on this pattern is a faint, light blue grid that recedes into the distance, creating a sense of depth and perspective.

Section 2

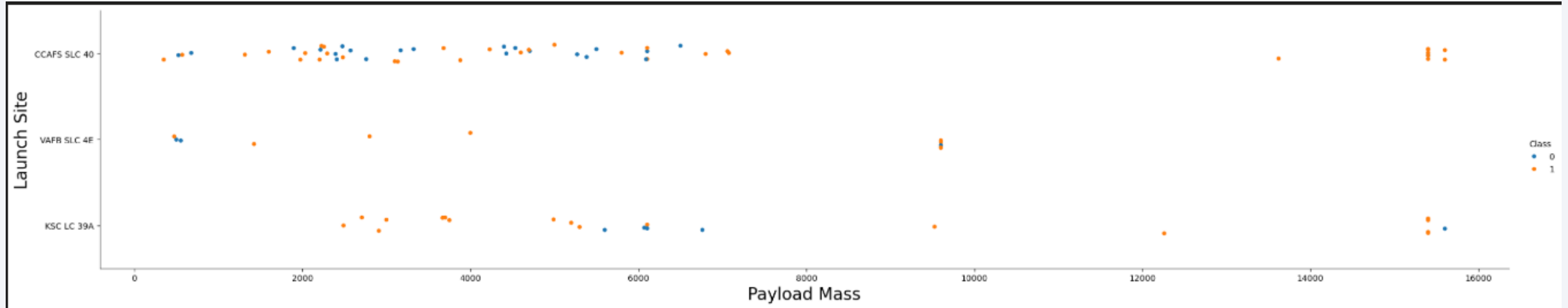
Insights drawn from EDA

Flight Number vs. Launch Site



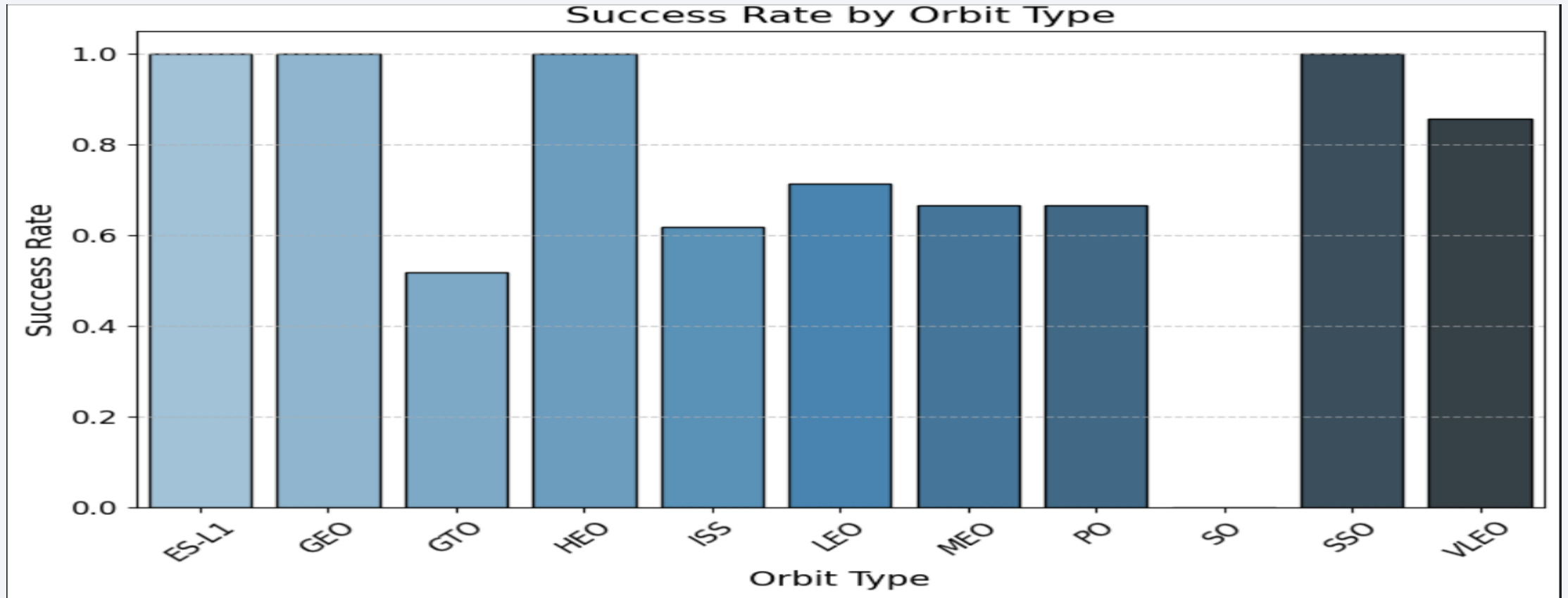
The most launch attempts was in the Launch site: CCAFS SLC 40, followed by KSC LC 39A and ended by VAFB SLC 4E. And there was most successfully launch in the launch site CCAFS SLC 40 than the others.

Payload vs. Launch Site



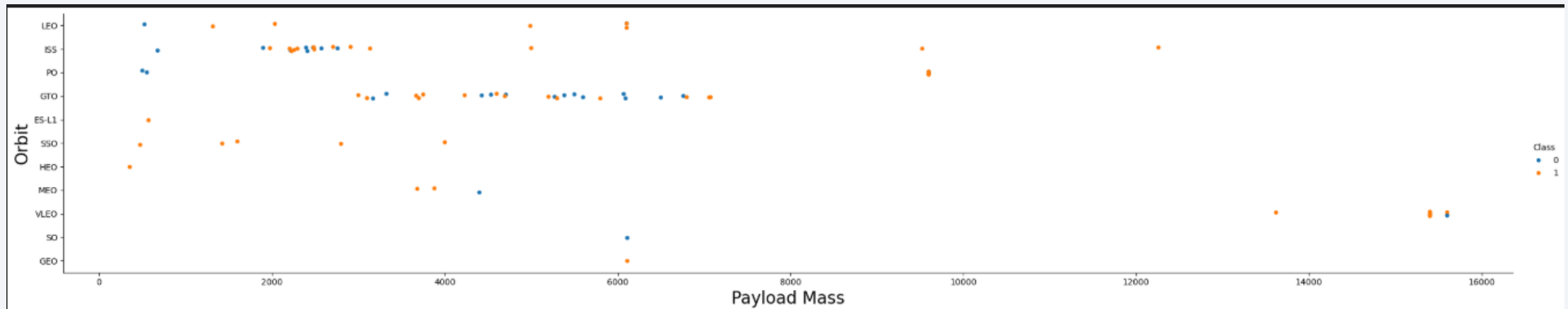
If you observe Payload Mass Vs. Launch Site scatter point chart you will find for the VAFB-SLC launchsite there are no rockets launched for heavypayload mass(greater than 10000)

Success Rate vs. Orbit Type



The orbits which have the highest success rate: **ES-L1, GEO, HEO, SSO**

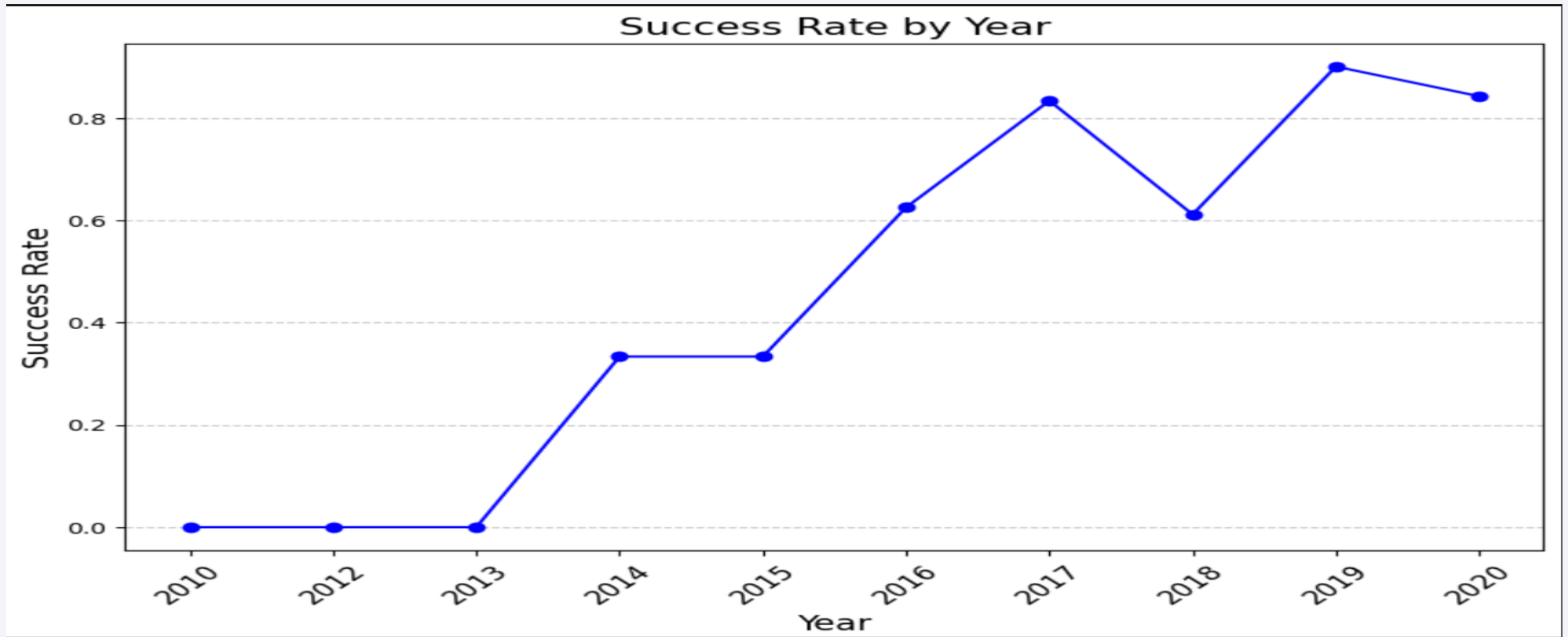
Payload vs. Orbit Type



With heavy payloads the successful landing or positive landing rate are more for Polar, LEO and ISS.

However, for GTO, it's difficult to distinguish between successful and unsuccessful landings as both outcomes are present.

Launch Success Yearly Trend



You can observe that the success rate since 2013 kept increasing till 2020

All Launch Site Names

```
%sql SELECT DISTINCT "Launch_Site" FROM SPACEXTABLE;
```

```
* sqlite:///my\_data1.db
```

```
Done.
```

Launch_Site
CCAFS LC-40
VAFB SLC-4E
KSC LC-39A
CCAFS SLC-40

There is four launch sites as shown as query result

Launch Site Names Begin with 'CCA'

```
%sql SELECT * FROM SPACEXTABLE WHERE "Launch_Site" LIKE 'CCA%' LIMIT 5;
```

[12] Python

... * [sqlite:///my_data1.db](#)

Done.

...

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS_KG	Orbit	Customer	Mission_Outcome	Landing
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	
2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	

This query give the name of launch site that begin by 'CCA'

Total Payload Mass

```
[17] %sql SELECT SUM("PAYLOAD_MASS__KG_") AS Total_Payload_Mass FROM SPACEXTABLE WHERE Customer='NASA (CRS)';  
... * sqlite:///my\_data1.db  
Done.  
... 

| Total_Payload_Mass |
|--------------------|
| 45596              |


```



The total payload mass carried by boosters launched by NASA (CRS)

Average Payload Mass by F9 v1.1

```
[18] %sql SELECT AVG("PAYLOAD_MASS__KG_") AS Average_Payload_Mass FROM SPACEXTABLE WHERE "Booster_Version" = 'F9 v1.1';  
... * sqlite:///my\_data1.db  
Done.  
... 

| Average_Payload_Mass |
|----------------------|
| 2928.4               |


```



The average payload mass carried by booster version F9 v1.1

First Successful Ground Landing Date

```
[19] %sql SELECT MIN("Date") AS First_Successful_Landing_Date FROM SPACEXTABLE WHERE "Landing_Outcome" = 'Success (grou
Python
... * sqlite:///my\_data1.db
Done.
... First_Successful_Landing_Date
      2015-12-22
```



The date of the first successful landing outcome on ground pad is December 22, 2015.

Successful Drone Ship Landing with Payload between 4000 and 6000

```
▶ %sql SELECT "Booster_Version" FROM SPACEXTABLE WHERE "Landing_Outcome" = 'Success (drone ship)' AND "PAYLOAD_MASS_
[20] Python
... * sqlite:///my\_data1.db
Done.
... Booster_Version
    F9 FT B1022
    F9 FT B1026
    F9 FT B1021.2
    F9 FT B1031.2
```

List of boosters which have successfully landed on drone ship and had payload mass greater than 4000 but less than 6000

Total Number of Successful and Failure Mission Outcomes

```
▶ %sql SELECT "Mission_Outcome", COUNT(*) AS Total_Count FROM SPACEXTABLE GROUP BY "Mission_Outcome";
[21]
... * sqlite:///my\_data1.db
Done.
...


| Mission_Outcome                  | Total_Count |
|----------------------------------|-------------|
| Failure (in flight)              | 1           |
| Success                          | 98          |
| Success                          | 1           |
| Success (payload status unclear) | 1           |


```

The total number of successful and failure mission outcomes

Boosters Carried Maximum Payload

```
▶ %sql SELECT "Booster_Version" FROM SPACEXTABLE WHERE "PAYLOAD_MASS__KG_" = (SELECT MAX("PAYLOAD_MASS__KG_") FROM SI
[22] Python

... * sqlite:///my\_data1.db
Done.

... Booster_Version
    F9 B5 B1048.4
    F9 B5 B1049.4
    F9 B5 B1051.3
    F9 B5 B1056.4
    F9 B5 B1048.5
    F9 B5 B1051.4
    F9 B5 B1049.5
    F9 B5 B1060.2
    F9 B5 B1058.3
    F9 B5 B1051.6
    F9 B5 B1060.3
    F9 B5 B1049.7
```

List of names of the booster which have carried the maximum payload mass

2015 Launch Records

```
▶ %sql SELECT substr("Date", 6, 2) AS Month, "Landing_Outcome", "Booster_Version", "Launch_Site" FROM SPACEXTABLE WHI
[23] Python
... * sqlite:///my\_data1.db
Done.
...


| Month | Landing_Outcome      | Booster_Version | Launch_Site |
|-------|----------------------|-----------------|-------------|
| 01    | Failure (drone ship) | F9 v1.1 B1012   | CCAFS LC-40 |
| 04    | Failure (drone ship) | F9 v1.1 B1015   | CCAFS LC-40 |


```

List of failed landing_outcomes in drone ship, their booster versions, and launch site names for in year 2015

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

```
▶ %sql SELECT "Landing_Outcome", COUNT(*) AS Outcome_Count FROM SPACEXTABLE WHERE "Date" BETWEEN '2010-06-04' AND '2017-03-20'
```

[24] Python

```
... * sqlite:///my\_data1.db  
Done.
```

```
... 
```

Landing_Outcome	Outcome_Count
No attempt	10
Success (drone ship)	5
Failure (drone ship)	5
Success (ground pad)	3
Controlled (ocean)	3
Uncontrolled (ocean)	2
Failure (parachute)	2
Precluded (drone ship)	1

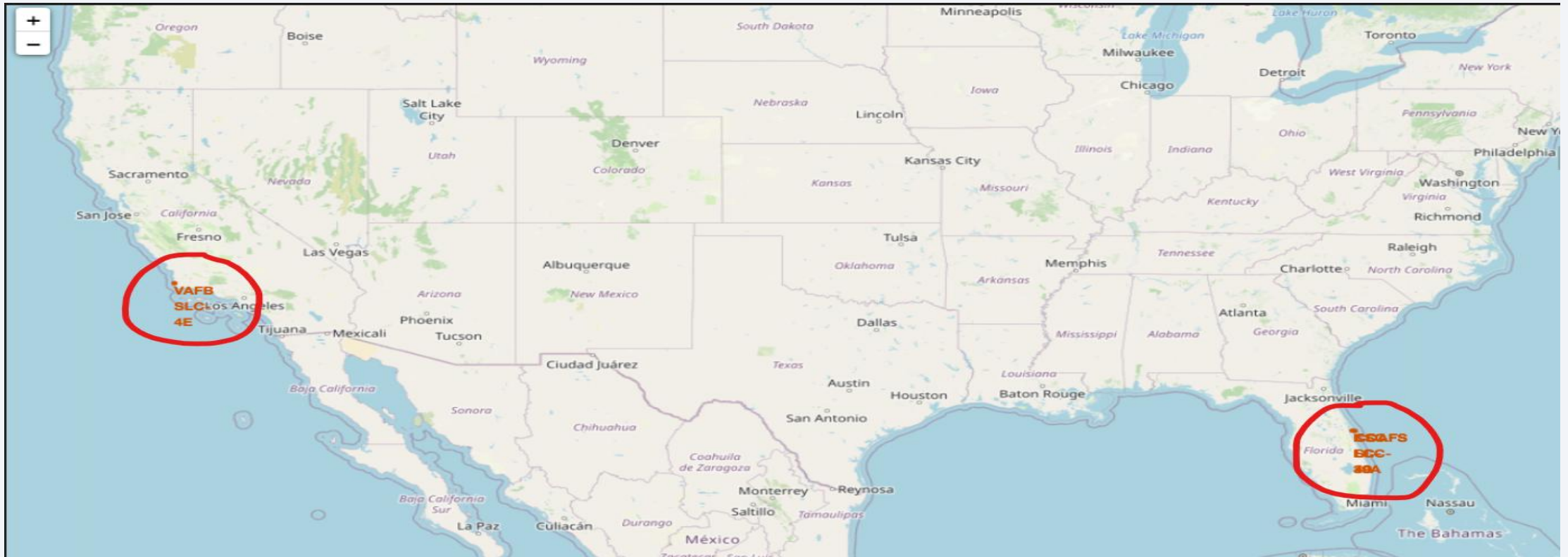
Rank of the count of landing outcomes (such as Failure (drone ship) or Success (ground pad)) between the date 2010-06-04 and 2017-03-20, in descending order

A satellite view of Earth from space, showing the curvature of the planet and city lights at night. The background is a deep blue gradient.

Section 3

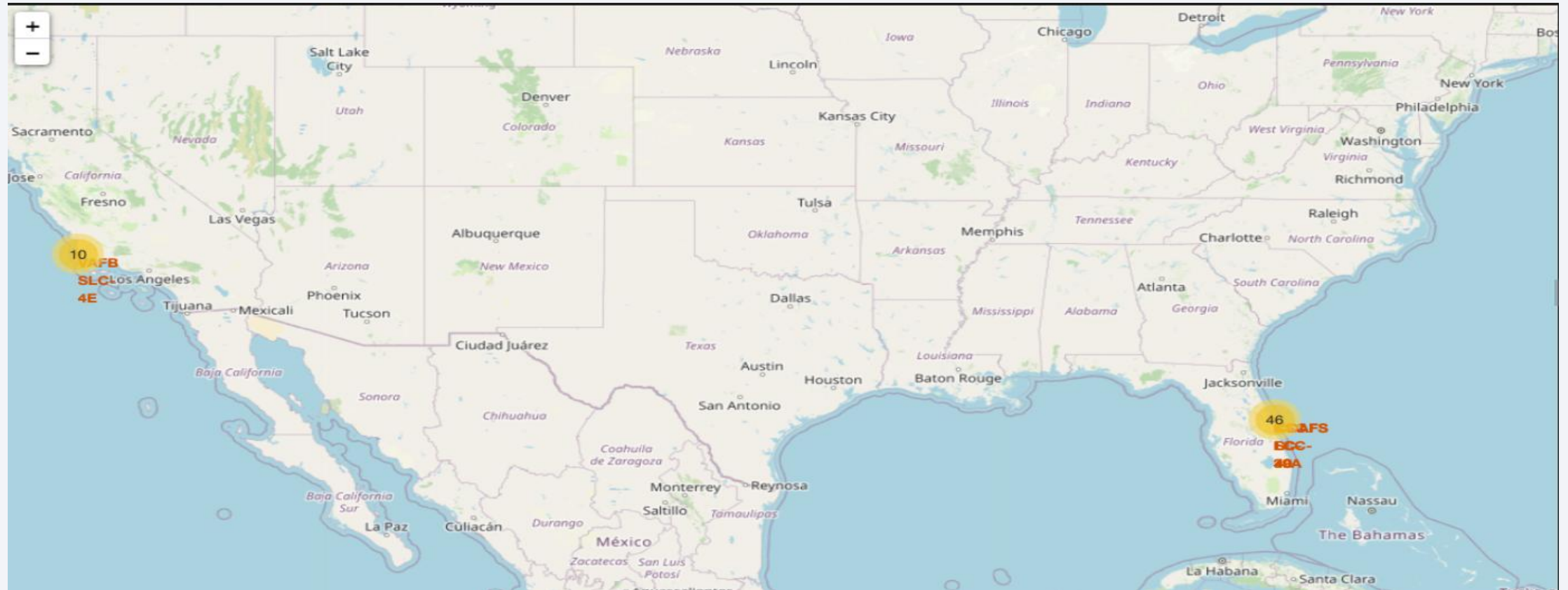
Launch Sites Proximities Analysis

View of different launch sites on the map



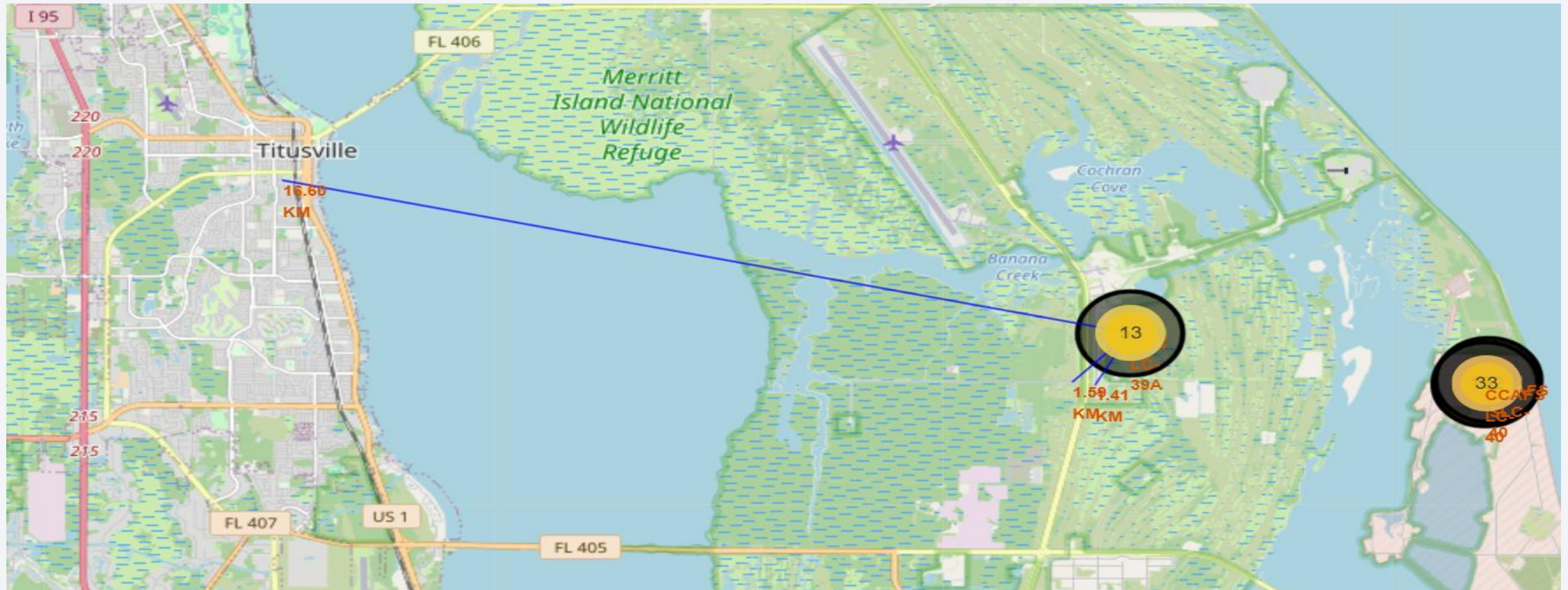
All launch sites in proximity to the Equator line and all launch sites in very close proximity to the coast as you can see.

Launch outcomes for each site

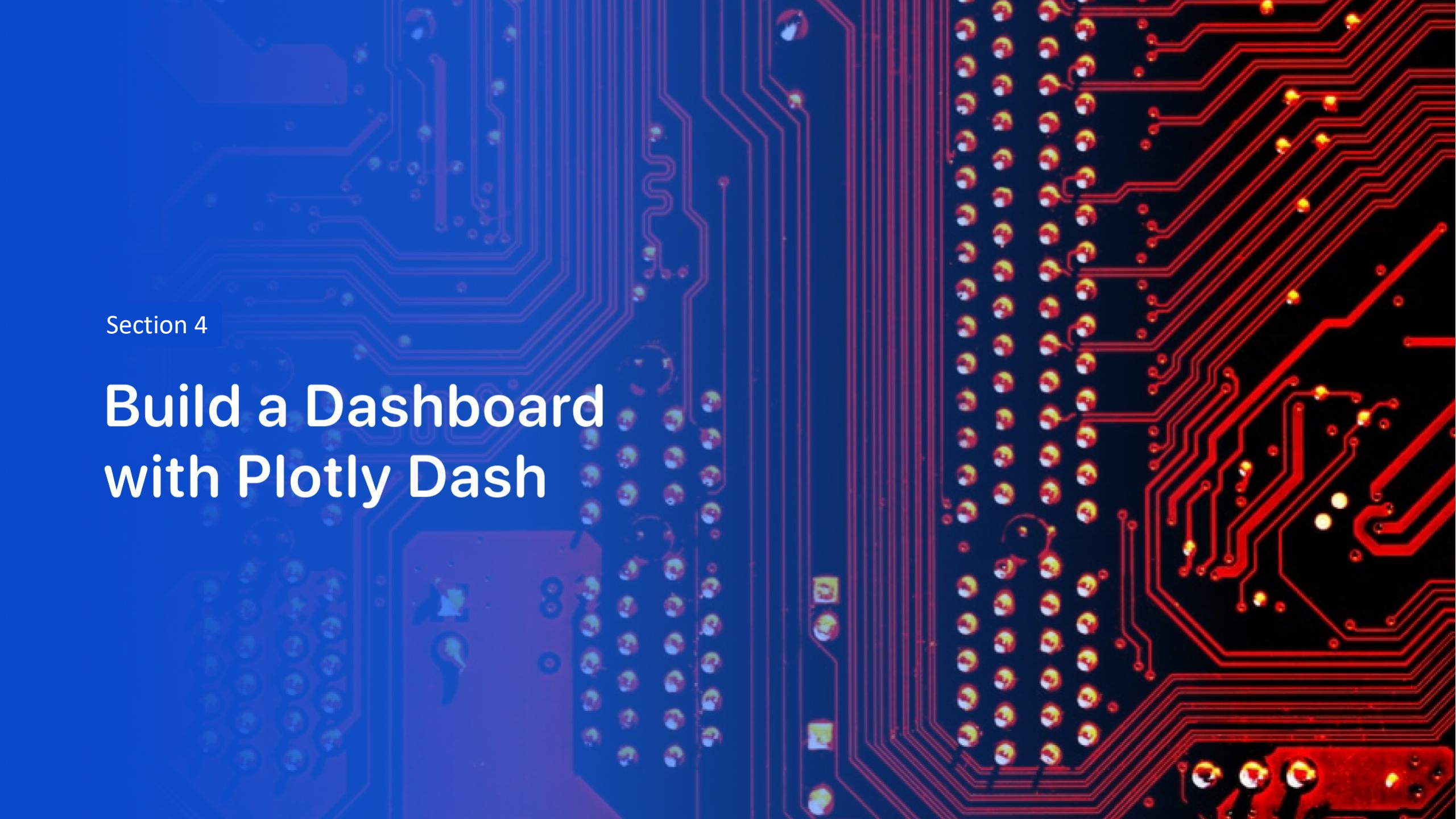


From the color-labeled markers in marker clusters on the bottom right, you should be able to easily identify 46, the launch sites which have relatively high success rates.

Proximities of launch sites



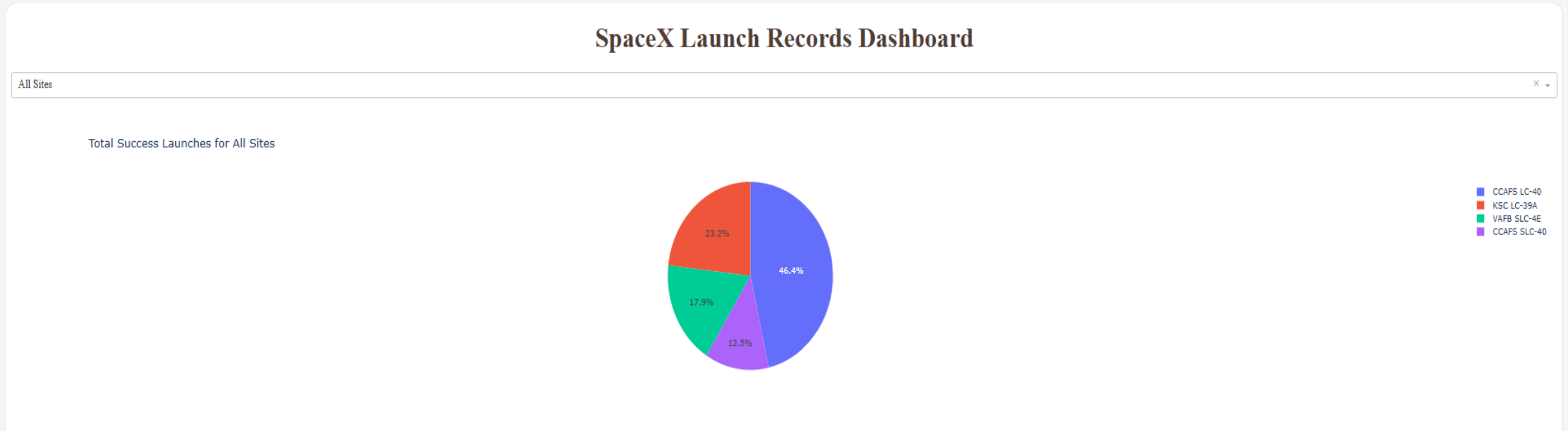
As you can notice the nearest city is Titusville at 16.6 km from the launch site and there are highways and railways so close.



Section 4

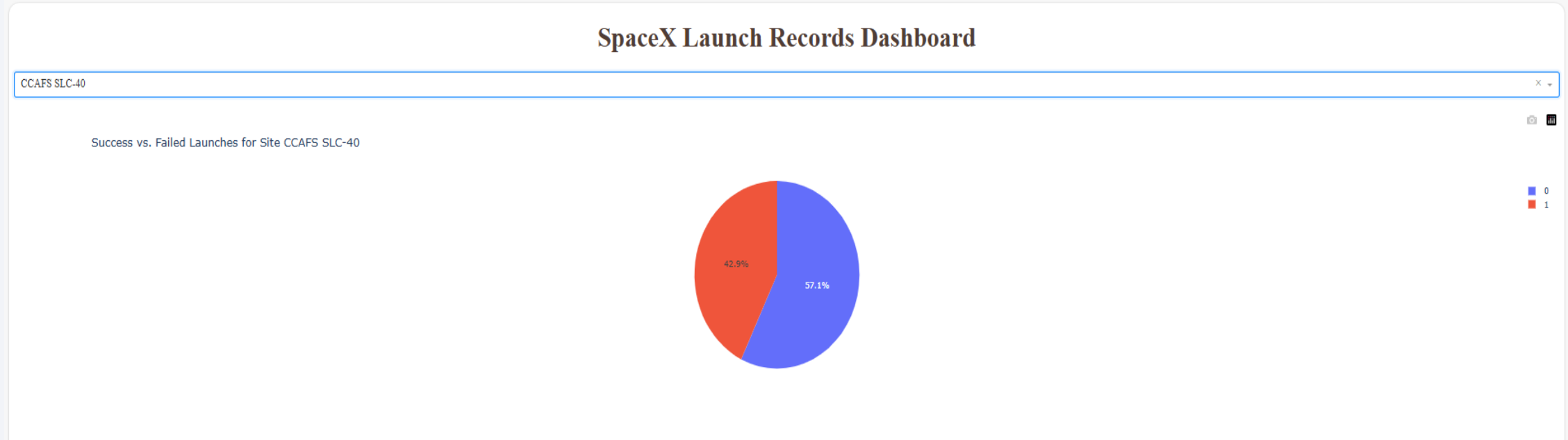
Build a Dashboard with Plotly Dash

Dashboard showing total success launch



As you can see CCAFS LC-40 have the most success launches with 46/4% percentage in blue, following by KSC LC6-29A in red.

Dashboard - Launch site with highest success ratio



Screenshot of the piechart for the launch site with highest launch success ratio: CCAFS SLC-40

<Dashboard Screenshot 3>



With low payload range, there is more attempts so more success rate (top image). However in the high payload range (bottom image) the success is very little. The booster version V1 -1 have the largest success rate, etc.

Section 5

Predictive Analysis (Classification)

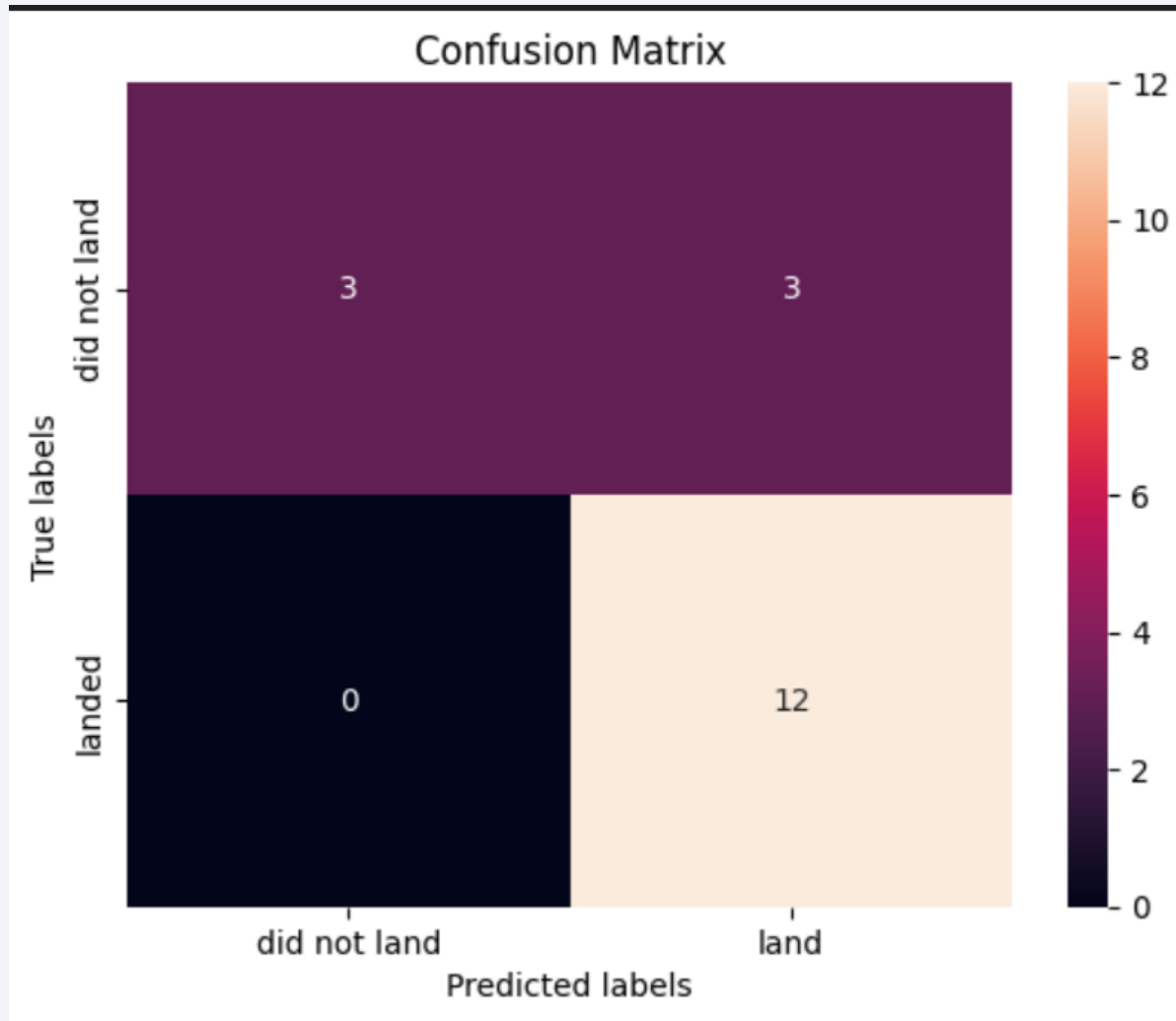
Classification Accuracy

•**Visualization:** Bar chart displaying accuracy percentages for each model:

- Logistic Regression: **84.64%**
- Decision Tree: **88.92%**
- SVM: **84.82 %**
- K Nearest Neighbors: **84.82%**

•**Conclusion:** Decision Tree Classifier achieved the highest classification accuracy at 88.92%, making it the best-performing model.

Confusion Matrix



This confusion matrix is Decision Tree hers. It shows that it performed 12 True Positive, 3 False Negative, 0 False Positive and 3 True Negative. That is to say it only make mistake by predicted three success land instead of predict fail.

Conclusions

Key Achievements:

- Developed advanced models, achieving up to 88% accuracy (Decision Tree).
- Designed interactive dashboards showcasing data insights effectively.
- Created a detailed Folium map for spatial analysis.

Practical Applications:

- Predictive modeling techniques applicable to various industries.
- Interactive tools empower data-driven decision-making.

Final Thoughts:

- The project demonstrates mastery of data science concepts and tools.
- Skills gained are directly transferable to real-world scenarios.

Appendix

Python code sample

Linear Regression Implementation:

Python

 Copier

```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X_train, y_train)
predictions = model.predict(X_test)
```

Folium Map Creation:

Python

 Copier

```
import folium
map = folium.Map(location=[latitude, longitude], zoom_start=10)
folium.Marker([lat1, long1], tooltip='Marker 1').add_to(map)
map.save("map.html")
```


Thank you!

